

SmartLib

Source Code Submission

SCSM2223 : Cross-Platform Application Development

Group 4 : Polar Bear

Semester 2, 2025/2026

Name	Role	Matric
Chang Kai Yuan	UI Lead	A24MJ5069
Jeffrey Tan Zhi Yao	Backend Lead	A24MJ5042
Ahmed Samir	Security / Testing Lead	A24MJ3021
Abdelrahman Abdelkerim	Documentation Lead	A24MJ3025

Table of Contents

1. **Database schema**
`backend/database/schema.sql`
2. **Backend: Slim entry point**
`backend/public/index.php`
3. **Backend: PDO connection (db.php)**
`backend/src/db.php`
4. **Backend: JSON helpers**
`backend/src/helpers.php`
5. **Backend: JWT middleware**
`backend/src/middleware/JwtMiddleware.php`
6. **Backend: API routes (all endpoints)**
`backend/src/routes.php`
7. **Frontend: Vue entry point**
`frontend/src/main.js`
8. **Frontend: Vue Router + guards**
`frontend/src/router/index.js`
9. **Frontend: Axios client + JWT interceptor**
`frontend/src/api/http.js`
10. **Frontend: Reactive auth state**
`frontend/src/api/auth.js`
11. **Frontend: Root component (App.vue)**
`frontend/src/App.vue`
12. **Frontend: Book Catalogue view**
`frontend/src/views/BookListView.vue`
13. **Frontend: Borrow view**
`frontend/src/views/BorrowView.vue`
14. **Frontend: My Borrows / Return view**
`frontend/src/views/ReturnView.vue`
15. **Frontend: Admin CRUD view**
`frontend/src/views/AdminView.vue`
16. **Frontend: Login view**
`frontend/src/views/LoginView.vue`
17. **Frontend: Register view**
`frontend/src/views/RegisterView.vue`
18. **Frontend: Change Password view**
`frontend/src/views/ChangePasswordView.vue`
19. **Frontend: Book Card component**
`frontend/src/components/BookCard.vue`
20. **Frontend: Borrow Form component**
`frontend/src/components/BorrowForm.vue`
21. **Frontend: Search Bar component**
`frontend/src/components/SearchBar.vue`

22. Frontend: Loading Spinner component

`frontend/src/components/LoadingSpinner.vue`

Note: configuration files (composer.json, package.json, vite.config.js, .htaccess, .env.example) are deliberately excluded. They are standard framework boilerplate and would inflate similarity scores without representing original work.

1. Database schema

Path: backend/database/schema.sql Lines: 74

```
1 | -- =====
2 | -- SmartLib – Database Schema
3 | -- Run once in phpMyAdmin (or `mysql -u root smartlib < schema.sql`)
4 | -- after creating the empty `smartlib` database.
5 | -- =====
6 |
7 | CREATE DATABASE IF NOT EXISTS smartlib
8 |   CHARACTER SET utf8mb4
9 |   COLLATE utf8mb4_unicode_ci;
10 | USE smartlib;
11 |
12 | -- Idempotent reset (safe in dev – DO NOT run on production data)
13 | DROP TABLE IF EXISTS borrow_records;
14 | DROP TABLE IF EXISTS books;
15 | DROP TABLE IF EXISTS members;
16 |
17 | -- -----
18 | -- books – catalogue
19 | -- -----
20 | CREATE TABLE books (
21 |   id          INT AUTO_INCREMENT PRIMARY KEY,
22 |   title       VARCHAR(255) NOT NULL,
23 |   author      VARCHAR(255) NOT NULL,
24 |   isbn        CHAR(13) UNIQUE,
25 |   category    VARCHAR(100),
26 |   stock       INT NOT NULL DEFAULT 1,
27 |   available_count INT NOT NULL DEFAULT 1,
28 |   created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
29 |   INDEX idx_title (title),
30 |   INDEX idx_category (category),
31 |   CHECK (stock >= 0),
32 |   CHECK (available_count >= 0),
33 |   CHECK (available_count <= stock)
34 | ) ENGINE=InnoDB;
35 |
36 | -- -----
37 | -- members – users (students + admins)
38 | -- -----
39 | CREATE TABLE members (
40 |   id          INT AUTO_INCREMENT PRIMARY KEY,
41 |   name        VARCHAR(255) NOT NULL,
42 |   email       VARCHAR(255) UNIQUE NOT NULL,
43 |   password_hash VARCHAR(255) NOT NULL, -- bcrypt via password_hash()
44 |   role        ENUM('student', 'admin') NOT NULL DEFAULT 'student',
45 |   created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
46 | ) ENGINE=InnoDB;
47 |
48 | -- -----
49 | -- borrow_records – borrow/return history
50 | -- -----
51 | CREATE TABLE borrow_records (
52 |   id          INT AUTO_INCREMENT PRIMARY KEY,
53 |   member_id   INT NOT NULL,
54 |   book_id     INT NOT NULL,
55 |   borrow_date DATE NOT NULL,
56 |   due_date    DATE NOT NULL, -- borrow_date + BORROW_DAYS
57 |   return_date DATE DEFAULT NULL, -- NULL = still active
58 |   status      ENUM('active', 'returned', 'overdue') NOT NULL DEFAULT 'active',
59 |   FOREIGN KEY (member_id) REFERENCES members(id) ON DELETE CASCADE,
60 |   FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE RESTRICT,
61 |   INDEX idx_member (member_id),
62 |   INDEX idx_book (book_id),
63 |   INDEX idx_status (status)
64 | ) ENGINE=InnoDB;
65 |
66 | -- -----
67 | -- Optional helper: a daily scheduled event would flip stale
68 | -- 'active' records past due_date to 'overdue'. For the course
69 | -- project, this can run on demand instead:
```

```
70 | --  
71 | -- UPDATE borrow_records  
72 | --     SET status = 'overdue'  
73 | --     WHERE status = 'active' AND due_date < CURDATE();  
74 | -- -----
```

2. Backend: Slim entry point

Path: backend/public/index.php Lines: 75

[illegible]

```
70 | });  
71 |  
72 | // ■■ 5. Register routes  
73 | require __DIR__ . '/../src/routes.php';  
74 |  
75 | $app->run();
```

3. Backend: PDO connection (db.php)

Path: backend/src/db.php Lines: 35

```
1 | <?php
2 | /**
3 |  * SmartLib – PDO database singleton.
4 |  *
5 |  * Uses env vars loaded by phpdotenv in public/index.php.
6 |  * All queries elsewhere must use prepared statements (PDO::prepare + execute).
7 |  */
8 |
9 | declare(strict_types=1);
10 |
11 | function getDB(): PDO
12 | {
13 |     static $pdo = null;
14 |
15 |     if ($pdo !== null) {
16 |         return $pdo;
17 |     }
18 |
19 |     $host = $_ENV['DB_HOST'] ?? 'localhost';
20 |     $port = $_ENV['DB_PORT'] ?? '3306';
21 |     $name = $_ENV['DB_NAME'] ?? 'smartlib';
22 |     $user = $_ENV['DB_USER'] ?? 'root';
23 |     $pass = $_ENV['DB_PASS'] ?? '';
24 |
25 |     $dsn = "mysql:host={$host};port={$port};dbname={$name};charset=utf8mb4";
26 |
27 |     $pdo = new PDO($dsn, $user, $pass, [
28 |         PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
29 |         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
30 |         // Use real prepared statements (not emulated) for true SQLi protection
31 |         PDO::ATTR_EMULATE_PREPARES => false,
32 |     ]);
33 |
34 |     return $pdo;
35 | }
```

4. Backend: JSON helpers

Path: backend/src/helpers.php Lines: 43

```
1 | <?php
2 | /**
3 |  * SmartLib – Shared helpers for writing JSON responses.
4 |  *
5 |  * Keeps every route consistent with the standard response shape:
6 |  *   { status, code, message?, data? }
7 |  */
8 |
9 | declare(strict_types=1);
10 |
11 | use Psr\Http\Message\ResponseInterface as Response;
12 |
13 | /**
14 |  * Write a JSON success response.
15 |  *
16 |  * @param Response $rs
17 |  * @param array<string,mixed> $payload Extra fields merged into the response
18 |  * @param int $status HTTP status code (default 200)
19 |  */
20 | function jsonSuccess(Response $rs, array $payload = [], int $status = 200): Response
21 | {
22 |     $body = array_merge(['status' => 'success', 'code' => $status], $payload);
23 |     $rs->getBody()->write(json_encode($body));
24 |     return $rs
25 |         ->withHeader('Content-Type', 'application/json')
26 |         ->withStatus($status);
27 | }
28 |
29 | /**
30 |  * Write a JSON error response.
31 |  */
32 | function jsonError(Response $rs, string $message, int $status = 400): Response
33 | {
34 |     $body = [
35 |         'status' => 'error',
36 |         'code' => $status,
37 |         'message' => $message,
38 |     ];
39 |     $rs->getBody()->write(json_encode($body));
40 |     return $rs
41 |         ->withHeader('Content-Type', 'application/json')
42 |         ->withStatus($status);
43 | }
```

5. Backend: JWT middleware

Path: backend/src/middleware/JwtMiddleware.php Lines: 87

```
1 | <?php
2 | /**
3 |  * SmartLib – JWT authentication middleware.
4 |  *
5 |  * - Reads the `Authorization: Bearer <token>` header.
6 |  * - Verifies the signature with the secret from .env.
7 |  * - On success, attaches the decoded payload to the request as the `user` attribute.
8 |  * - On failure, returns 401 (missing/invalid) or 403 (wrong role).
9 |  *
10 |  * Usage in routes.php:
11 |  *   $app->post('/api/books', [BookController::class, 'create'])
12 |  *     ->add(new JwtMiddleware('admin')); // require admin role
13 |  *
14 |  *   $app->post('/api/borrow', $borrowHandler)
15 |  *     ->add(new JwtMiddleware()); // any authenticated user
16 |  */
17 |
18 | declare(strict_types=1);
19 |
20 | namespace SmartLib\Middleware;
21 |
22 | use Firebase\JWT\JWT;
23 | use Firebase\JWT\Key;
24 | use Psr\Http\Message\ResponseInterface as Response;
25 | use Psr\Http\Message\ServerRequestInterface as Request;
26 | use Psr\Http\Server\MiddlewareInterface;
27 | use Psr\Http\Server\RequestHandlerInterface as Handler;
28 | use Slim\Psr7\Factory\ResponseFactory;
29 | use Throwable;
30 |
31 | class JwtMiddleware implements MiddlewareInterface
32 | {
33 |     /**
34 |      * @param string|null $requiredRole 'admin' to restrict to admins, null for any logged-in user
35 |      */
36 |     public function __construct(private ?string $requiredRole = null)
37 |     {
38 |     }
39 |
40 |     public function process(Request $request, Handler $handler): Response
41 |     {
42 |         $header = $request->getHeaderLine('Authorization');
43 |
44 |         if (!$header || !preg_match('/^Bearer\s+(.+)$/i', $header, $matches)) {
45 |             return $this->respond(401, 'Missing or malformed Authorization header');
46 |         }
47 |
48 |         $token = $matches[1];
49 |         $secret = $_ENV['JWT_SECRET'] ?? '';
50 |         $algo = $_ENV['JWT_ALGO'] ?? 'HS256';
51 |
52 |         if ($secret === '' || $secret === 'replace-me-with-a-long-random-string') {
53 |             // Refuse to run with an unconfigured secret – fail closed
54 |             return $this->respond(500, 'Server JWT secret is not configured');
55 |         }
56 |
57 |         try {
58 |             $decoded = JWT::decode($token, new Key($secret, $algo));
59 |         } catch (Throwable $e) {
60 |             return $this->respond(401, 'Invalid or expired token');
61 |         }
62 |
63 |         // Role check (if route demands a specific role)
64 |         if ($this->requiredRole !== null && ($decoded->role ?? null) !== $this->requiredRole) {
65 |             return $this->respond(403, 'Forbidden – insufficient privileges');
66 |         }
67 |
68 |         // Attach decoded payload to the request for downstream handlers
69 |         $request = $request
```

```

70 |         ->withAttribute('user', (array) $decoded)
71 |         ->withAttribute('user_id', $decoded->id ?? null)
72 |         ->withAttribute('user_role', $decoded->role ?? null);
73 |
74 |     return $handler->handle($request);
75 | }
76 |
77 | private function respond(int $status, string $message): Response
78 | {
79 |     $response = (new ResponseFactory())->createResponse($status);
80 |     $response->getBody()->write(json_encode([
81 |         'status' => 'error',
82 |         'code'   => $status,
83 |         'message' => $message,
84 |     ]));
85 |     return $response->withHeader('Content-Type', 'application/json');
86 | }
87 | }

```

6. Backend: API routes (all endpoints)

Path: backend/src/routes.php Lines: 461

```
1 | <?php
2 | /**
3 |  * SmartLib – API route definitions (10 endpoints).
4 |  *
5 |  * Public routes:
6 |  *   GET /api/books          List available books
7 |  *   GET /api/books/{id}     Get one book
8 |  *   POST /api/members/register Register a student
9 |  *   POST /api/members/login  Log in, get JWT
10 |  *
11 |  * Protected routes (JWT required):
12 |  *   POST /api/books          Create book          [Admin]
13 |  *   PUT /api/books/{id}      Update book          [Admin]
14 |  *   DELETE /api/books/{id}   Delete book          [Admin]
15 |  *   POST /api/borrow         Borrow a book        [Any authed user]
16 |  *   POST /api/return/{id}    Return a borrow record [Owner or Admin]
17 |  *   GET /api/members         List all members      [Admin]
18 |  *
19 |  * Response shape: { status, code, message?, data? }.
20 |  * Every error path uses jsonError() and returns the matching HTTP status.
21 |  */
22 |
23 | declare(strict_types=1);
24 |
25 | use Firebase\JWT\JWT;
26 | use Psr\Http\Message\ResponseInterface as Response;
27 | use Psr\Http\Message\ServerRequestInterface as Request;
28 | use SmartLib\Middleware\JwtMiddleware;
29 |
30 | require __DIR__ . '/helpers.php';
31 | require __DIR__ . '/middleware/JwtMiddleware.php';
32 |
33 | /** @var \Slim\App $app (provided by public/index.php) */
34 | $pdo = getDB();
35 |
36 | // =====
37 | // 1. GET /api/books – list available books (with optional ?search=)
38 | // =====
39 | $app->get('/api/books', function (Request $rq, Response $rs) use ($pdo): Response {
40 |     $params = $rq->getQueryParams();
41 |     $search = trim($params['search'] ?? '');
42 |
43 |     if ($search !== '') {
44 |         $stmt = $pdo->prepare(
45 |             'SELECT * FROM books
46 |              WHERE (title LIKE :q OR author LIKE :q OR category LIKE :q)
47 |              ORDER BY title ASC'
48 |         );
49 |         $stmt->execute([':q' => '%' . $search . '%']);
50 |     } else {
51 |         $stmt = $pdo->prepare('SELECT * FROM books ORDER BY title ASC');
52 |         $stmt->execute();
53 |     }
54 |
55 |     return jsonSuccess($rs, ['data' => $stmt->fetchAll()]);
56 | });
57 |
58 | // =====
59 | // 2. GET /api/books/{id} – fetch single book
60 | // =====
61 | $app->get('/api/books/{id}', function (Request $rq, Response $rs, array $args) use ($pdo): Response
62 | {
63 |     $id = (int) $args['id'];
64 |
65 |     $stmt = $pdo->prepare('SELECT * FROM books WHERE id = ?');
66 |     $stmt->execute([$id]);
67 |     $book = $stmt->fetch();
68 |
69 |     if (!$book) {
```

```

69 |         return jsonError($rs, 'Book not found', 404);
70 |     }
71 |
72 |     return jsonSuccess($rs, ['data' => $book]);
73 | });
74 |
75 | // =====
76 | // 3. POST /api/books – create a new book (Admin only)
77 | // =====
78 | $app->post('/api/books', function (Request $rq, Response $rs) use ($pdo): Response {
79 |     $body = $rq->getParsedBody() ?? [];
80 |     $title = trim($body['title'] ?? '');
81 |     $author = trim($body['author'] ?? '');
82 |     $isbn = trim($body['isbn'] ?? '');
83 |     $category = trim($body['category'] ?? '');
84 |     $stock = (int) ($body['stock'] ?? 0);
85 |
86 |     if ($title === '' || $author === '' || $stock < 1) {
87 |         return jsonError($rs, 'Missing required fields: title, author, stock (>=1)', 400);
88 |     }
89 |
90 |     try {
91 |         $stmt = $pdo->prepare(
92 |             'INSERT INTO books (title, author, isbn, category, stock, available_count)
93 |             VALUES (?, ?, ?, ?, ?, ?)'
94 |         );
95 |         $stmt->execute([
96 |             $title,
97 |             $author,
98 |             $isbn !== '' ? $isbn : null,
99 |             $category !== '' ? $category : null,
100 |             $stock,
101 |             $stock,
102 |         ]);
103 |     } catch (PDOException $e) {
104 |         // 1062 = duplicate ISBN
105 |         if ($e->errorInfo[1] ?? null === 1062) {
106 |             return jsonError($rs, 'A book with this ISBN already exists', 409);
107 |         }
108 |         throw $e;
109 |     }
110 |
111 |     return jsonSuccess($rs, ['id' => (int) $pdo->lastInsertId()], 201);
112 | }->add(new JwtMiddleware('admin'));
113 |
114 | // =====
115 | // 4. PUT /api/books/{id} – update a book (Admin only)
116 | // =====
117 | $app->put('/api/books/{id}', function (Request $rq, Response $rs, array $args) use ($pdo): Response
118 | {
119 |     $id = (int) $args['id'];
120 |     $body = $rq->getParsedBody() ?? [];
121 |
122 |     // Confirm book exists first
123 |     $stmt = $pdo->prepare('SELECT * FROM books WHERE id = ?');
124 |     $stmt->execute([$id]);
125 |     $book = $stmt->fetch();
126 |     if (!$book) {
127 |         return jsonError($rs, 'Book not found', 404);
128 |     }
129 |
130 |     // Merge submitted fields over existing record
131 |     $title = trim($body['title'] ?? $book['title']);
132 |     $author = trim($body['author'] ?? $book['author']);
133 |     $isbn = $body['isbn'] ?? $book['isbn'];
134 |     $category = $body['category'] ?? $book['category'];
135 |     $stock = isset($body['stock']) ? (int) $body['stock'] : (int) $book['stock'];
136 |
137 |     if ($title === '' || $author === '' || $stock < 0) {
138 |         return jsonError($rs, 'Invalid fields: title and author required, stock >= 0', 400);
139 |     }
140 |
141 |     // Keep available_count in sync if stock changed

```

```

141 |     $borrowedNow = (int) $book['stock'] - (int) $book['available_count'];
142 |     $newAvailable = max(0, $stock - $borrowedNow);
143 |
144 |     $stmt = $pdo->prepare(
145 |         'UPDATE books
146 |             SET title = ?, author = ?, isbn = ?, category = ?, stock = ?, available_count = ?
147 |             WHERE id = ?'
148 |     );
149 |     $stmt->execute([$title, $author, $isbn, $category, $stock, $newAvailable, $id]);
150 |
151 |     return jsonSuccess($rs);
152 | })->add(new JwtMiddleware('admin'));
153 |
154 | // =====
155 | // 5. DELETE /api/books/{id} – remove a book (Admin only)
156 | // =====
157 | $app->delete('/api/books/{id}', function (Request $rq, Response $rs, array $args) use ($pdo): Response {
158 |     $id = (int) $args['id'];
159 |
160 |     // Block deletion if any active borrow still references this book
161 |     $stmt = $pdo->prepare(
162 |         "SELECT COUNT(*) FROM borrow_records WHERE book_id = ? AND status = 'active'"
163 |     );
164 |     $stmt->execute([$id]);
165 |     if ((int) $stmt->fetchColumn() > 0) {
166 |         return jsonError($rs, 'Cannot delete – book has active borrows', 409);
167 |     }
168 |
169 |     $stmt = $pdo->prepare('DELETE FROM books WHERE id = ?');
170 |     $stmt->execute([$id]);
171 |
172 |     if ($stmt->rowCount() === 0) {
173 |         return jsonError($rs, 'Book not found', 404);
174 |     }
175 |
176 |     return jsonSuccess($rs);
177 | })->add(new JwtMiddleware('admin'));
178 |
179 | // =====
180 | // 6. POST /api/members/register – create a student account
181 | // =====
182 | $app->post('/api/members/register', function (Request $rq, Response $rs) use ($pdo): Response {
183 |     $body = $rq->getParsedBody() ?? [];
184 |     $name = trim($body['name'] ?? '');
185 |     $email = strtolower(trim($body['email'] ?? ''));
186 |     $pass = (string) ($body['password'] ?? '');
187 |
188 |     if ($name === '' || !filter_var($email, FILTER_VALIDATE_EMAIL) || strlen($pass) < 8) {
189 |         return jsonError($rs, 'Invalid input – name required, valid email, password >= 8 chars', 400);
190 |     }
191 |
192 |     // Check duplicate email
193 |     $stmt = $pdo->prepare('SELECT id FROM members WHERE email = ?');
194 |     $stmt->execute([$email]);
195 |     if ($stmt->fetch()) {
196 |         return jsonError($rs, 'Email already registered', 409);
197 |     }
198 |
199 |     $hash = password_hash($pass, PASSWORD_BCRYPT);
200 |
201 |     $stmt = $pdo->prepare(
202 |         "INSERT INTO members (name, email, password_hash, role)
203 |         VALUES (?, ?, ?, 'student')"
204 |     );
205 |     $stmt->execute([$name, $email, $hash]);
206 |
207 |     return jsonSuccess($rs, ['id' => (int) $pdo->lastInsertId()], 201);
208 | });
209 |
210 | // =====
211 | // 7. POST /api/members/login – issue JWT

```

```

212 | // =====
213 | $app->post('/api/members/login', function (Request $rq, Response $rs) use ($pdo): Response {
214 |     $body = $rq->getParsedBody() ?? [];
215 |     $email = strtolower(trim($body['email'] ?? ''));
216 |     $pass = (string) ($body['password'] ?? '');
217 |
218 |     if ($email === '' || $pass === '') {
219 |         return jsonError($rs, 'Email and password required', 400);
220 |     }
221 |
222 |     $stmt = $pdo->prepare('SELECT * FROM members WHERE email = ?');
223 |     $stmt->execute([$email]);
224 |     $member = $stmt->fetch();
225 |
226 |     // Use the same generic error for both "no such user" and "wrong password"
227 |     // so we don't leak which emails are registered.
228 |     if (!$member || !password_verify($pass, $member['password_hash'])) {
229 |         return jsonError($rs, 'Invalid credentials', 401);
230 |     }
231 |
232 |     $secret = $_ENV['JWT_SECRET'] ?? '';
233 |     $algo = $_ENV['JWT_ALGO'] ?? 'HS256';
234 |     $ttl = (int) ($_ENV['JWT_TTL'] ?? 3600);
235 |
236 |     $payload = [
237 |         'iss' => 'smartlib',
238 |         'iat' => time(),
239 |         'exp' => time() + $ttl,
240 |         'id' => (int) $member['id'],
241 |         'name' => $member['name'],
242 |         'role' => $member['role'],
243 |     ];
244 |
245 |     $token = JWT::encode($payload, $secret, $algo);
246 |
247 |     return jsonSuccess($rs, [
248 |         'token' => $token,
249 |         'user' => [
250 |             'id' => (int) $member['id'],
251 |             'name' => $member['name'],
252 |             'email' => $member['email'],
253 |             'role' => $member['role'],
254 |         ],
255 |     ]);
256 | });
257 |
258 | // =====
259 | // 8. POST /api/borrow – borrow a book (any authed user)
260 | // =====
261 | $app->post('/api/borrow', function (Request $rq, Response $rs) use ($pdo): Response {
262 |     $body = $rq->getParsedBody() ?? [];
263 |     $bookId = (int) ($body['book_id'] ?? 0);
264 |     $userId = (int) $rq->getAttribute('user_id');
265 |
266 |     if ($bookId < 1) {
267 |         return jsonError($rs, 'book_id is required', 400);
268 |     }
269 |
270 |     $borrowDays = (int) ($_ENV['BORROW_DAYS'] ?? 14);
271 |
272 |     // Wrap in a transaction so the stock decrement and the insert succeed together.
273 |     $pdo->beginTransaction();
274 |     try {
275 |         // Lock the row to prevent two concurrent borrows from over-issuing the last copy
276 |         $stmt = $pdo->prepare('SELECT * FROM books WHERE id = ? FOR UPDATE');
277 |         $stmt->execute([$bookId]);
278 |         $book = $stmt->fetch();
279 |
280 |         if (!$book) {
281 |             $pdo->rollBack();
282 |             return jsonError($rs, 'Book not found', 404);
283 |         }
284 |         if ((int) $book['available_count'] < 1) {

```

```

285 |         $pdo->rollBack();
286 |         return jsonResponse($rs, 'No copies available', 409);
287 |     }
288 |
289 |     // Block double-borrow of the same book by the same user
290 |     $stmt = $pdo->prepare(
291 |         "SELECT id FROM borrow_records
292 |         WHERE member_id = ? AND book_id = ? AND status = 'active'"
293 |     );
294 |     $stmt->execute([$userId, $bookId]);
295 |     if ($stmt->fetch()) {
296 |         $pdo->rollBack();
297 |         return jsonResponse($rs, 'You already have an active borrow for this book', 409);
298 |     }
299 |
300 |     $borrowDate = date('Y-m-d');
301 |     $dueDate     = date('Y-m-d', strtotime("+{$borrowDays} days"));
302 |
303 |     $stmt = $pdo->prepare(
304 |         "INSERT INTO borrow_records (member_id, book_id, borrow_date, due_date, status)
305 |         VALUES (?, ?, ?, ?, 'active')"
306 |     );
307 |     $stmt->execute([$userId, $bookId, $borrowDate, $dueDate]);
308 |     $recordId = (int) $pdo->lastInsertId();
309 |
310 |     $stmt = $pdo->prepare('UPDATE books SET available_count = available_count - 1 WHERE id = ?'
311 | );
312 |     $stmt->execute([$bookId]);
313 |
314 |     $pdo->commit();
315 | } catch (Throwable $e) {
316 |     $pdo->rollBack();
317 |     throw $e;
318 | }
319 |
320 | return jsonSuccess($rs, [
321 |     'record_id' => $recordId,
322 |     'due_date'  => $dueDate,
323 | ], 201);
324 | }->add(new JwtMiddleware());
325 |
326 | // =====
327 | // 9. POST /api/return/{id} – return a borrow record
328 | // =====
329 | $app->post('/api/return/{id}', function (Request $rq, Response $rs, array $args) use ($pdo): Respon
330 | se {
331 |     $recordId = (int) $args['id'];
332 |     $userId   = (int) $rq->getAttribute('user_id');
333 |     $role     = (string) $rq->getAttribute('user_role');
334 |
335 |     $pdo->beginTransaction();
336 |     try {
337 |         $stmt = $pdo->prepare('SELECT * FROM borrow_records WHERE id = ? FOR UPDATE');
338 |         $stmt->execute([$recordId]);
339 |         $record = $stmt->fetch();
340 |
341 |         if (!$record) {
342 |             $pdo->rollBack();
343 |             return jsonResponse($rs, 'Borrow record not found', 404);
344 |         }
345 |
346 |         // Only the borrower or an admin may return
347 |         if ((int) $record['member_id'] !== $userId && $role !== 'admin') {
348 |             $pdo->rollBack();
349 |             return jsonResponse($rs, 'Forbidden', 403);
350 |         }
351 |
352 |         if ($record['status'] !== 'active') {
353 |             $pdo->rollBack();
354 |             return jsonResponse($rs, 'This record is already returned', 409);
355 |         }
356 |
357 |         $returnDate = date('Y-m-d');

```

```

356 |         $stmt = $pdo->prepare(
357 |             "UPDATE borrow_records
358 |                 SET return_date = ?, status = 'returned'
359 |                 WHERE id = ?"
360 |         );
361 |         $stmt->execute([$returnDate, $recordId]);
362 |
363 |         $stmt = $pdo->prepare('UPDATE books SET available_count = available_count + 1 WHERE id = ?'
364 | );
365 |         $stmt->execute([$record['book_id']]);
366 |
367 |         $pdo->commit();
368 |     } catch (Throwable $e) {
369 |         $pdo->rollBack();
370 |         throw $e;
371 |     }
372 |     return jsonSuccess($rs, ['return_date' => $returnDate]);
373 | })->add(new JwtMiddleware());
374 |
375 | // =====
376 | // 10. GET /api/members – list all members (Admin only)
377 | // =====
378 | $app->get('/api/members', function (Request $rq, Response $rs) use ($pdo): Response {
379 |     // Never expose password_hash
380 |     $stmt = $pdo->prepare(
381 |         'SELECT id, name, email, role, created_at
382 |         FROM members
383 |         ORDER BY created_at DESC'
384 |     );
385 |     $stmt->execute();
386 |
387 |     return jsonSuccess($rs, ['data' => $stmt->fetchAll()]);
388 | })->add(new JwtMiddleware('admin'));
389 |
390 | // =====
391 | // POST /api/members/password – logged-in user changes their own password.
392 | //
393 | // Requires:
394 | //   - Valid JWT (any role)
395 | //   - current_password matches the stored bcrypt hash
396 | //   - new_password is at least 8 chars and different from current
397 | // =====
398 | $app->post('/api/members/password', function (Request $rq, Response $rs) use ($pdo): Response {
399 |     $userId = (int) $rq->getAttribute('user_id');
400 |     $body = $rq->getParsedBody() ?? [];
401 |     $current = (string) ($body['current_password'] ?? '');
402 |     $new = (string) ($body['new_password'] ?? '');
403 |
404 |     if ($current === '' || $new === '') {
405 |         return jsonError($rs, 'Both current_password and new_password are required', 400);
406 |     }
407 |     if (strlen($new) < 8) {
408 |         return jsonError($rs, 'New password must be at least 8 characters', 400);
409 |     }
410 |     if ($current === $new) {
411 |         return jsonError($rs, 'New password must be different from the current one', 400);
412 |     }
413 |
414 |     $stmt = $pdo->prepare('SELECT id, password_hash FROM members WHERE id = ?');
415 |     $stmt->execute([$userId]);
416 |     $member = $stmt->fetch();
417 |
418 |     if (!$member) {
419 |         return jsonError($rs, 'Account not found', 404);
420 |     }
421 |
422 |     // Constant-time check – protects against timing attacks.
423 |     if (!password_verify($current, $member['password_hash'])) {
424 |         return jsonError($rs, 'Current password is incorrect', 401);
425 |     }
426 |
427 |     $newHash = password_hash($new, PASSWORD_BCRYPT);

```

```

428 |     $stmt = $pdo->prepare('UPDATE members SET password_hash = ? WHERE id = ?');
429 |     $stmt->execute([$newHash, $userId]);
430 |
431 |     // NOTE: existing JWTs are still valid until they expire (stateless tokens).
432 |     // For a real production system you would add a token-blacklist or
433 |     // "password_changed_at" timestamp to invalidate older tokens. For this
434 |     // course project we accept the trade-off – documented in the security log.
435 |     return jsonSuccess($rs, ['message' => 'Password updated successfully']);
436 | }->add(new JwtMiddleware());
437 |
438 | // =====
439 | // Bonus: GET /api/borrow/me – list current user's borrow history (handy for ReturnView)
440 | // =====
441 | $app->get('/api/borrow/me', function (Request $rq, Response $rs) use ($pdo): Response {
442 |     $userId = (int) $rq->getAttribute('user_id');
443 |
444 |     $stmt = $pdo->prepare(
445 |         'SELECT br.*, b.title, b.author
446 |         FROM borrow_records br
447 |         JOIN books b ON b.id = br.book_id
448 |         WHERE br.member_id = ?
449 |         ORDER BY br.borrow_date DESC'
450 |     );
451 |     $stmt->execute([$userId]);
452 |
453 |     return jsonSuccess($rs, ['data' => $stmt->fetchAll()]);
454 | }->add(new JwtMiddleware());
455 |
456 | // =====
457 | // Health check – handy for verifying the server is up
458 | // =====
459 | $app->get('/', function (Request $rq, Response $rs): Response {
460 |     return jsonSuccess($rs, ['data' => ['service' => 'SmartLib API', 'version' => '1.0.0']]);
461 | });

```

7. Frontend: Vue entry point

Path: frontend/src/main.js Lines: 16

```
1 | /**
2 |  * SmartLib – Vue 3 entry point.
3 |  *
4 |  * Creates the app, wires the router, and configures Axios so every
5 |  * subsequent component just imports the shared client from `@/api/http`.
6 |  */
7 |
8 | import { createApp } from 'vue'
9 | import App from './App.vue'
10 | import router from './router'
11 | import './assets/main.css'
12 |
13 | // Make sure axios is configured BEFORE the app mounts.
14 | import './api/http.js'
15 |
16 | createApp(App).use(router).mount('#app')
```

8. Frontend: Vue Router + guards

Path: frontend/src/router/index.js Lines: 77

```
1 | /**
2 |  * Vue Router – defines every page in SmartLib and gates them
3 |  * with a global beforeEach guard that checks JWT + role claims.
4 |  */
5 |
6 | import { createRouter, createWebHistory } from 'vue-router'
7 |
8 | const routes = [
9 |   {
10 |     path: '/',
11 |     component: () => import('@/views/BookListView.vue'),
12 |     meta: { title: 'Catalogue' }
13 |   },
14 |   {
15 |     path: '/login',
16 |     component: () => import('@/views/LoginView.vue'),
17 |     meta: { title: 'Login', hideNav: true }
18 |   },
19 |   {
20 |     path: '/register',
21 |     component: () => import('@/views/RegisterView.vue'),
22 |     meta: { title: 'Register', hideNav: true }
23 |   },
24 |   {
25 |     path: '/borrow/:id',
26 |     component: () => import('@/views/BorrowView.vue'),
27 |     meta: { title: 'Borrow', requiresAuth: true }
28 |   },
29 |   {
30 |     path: '/my-borrows',
31 |     component: () => import('@/views/ReturnView.vue'),
32 |     meta: { title: 'My Borrows', requiresAuth: true }
33 |   },
34 |   {
35 |     path: '/admin',
36 |     component: () => import('@/views/AdminView.vue'),
37 |     meta: { title: 'Admin', requiresAuth: true, requiresAdmin: true }
38 |   },
39 |   {
40 |     path: '/account',
41 |     component: () => import('@/views/ChangePasswordView.vue'),
42 |     meta: { title: 'Change password', requiresAuth: true }
43 |   },
44 |   // Catch-all → home
45 |   { path: '/*', redirect: '/' }
46 | ]
47 |
48 | const router = createRouter({
49 |   history: createWebHistory(),
50 |   routes
51 | })
52 |
53 | // Decode the role out of the user object we stash at login time.
54 | // (We never *trust* this client-side – the backend still enforces it on every request.)
55 | function currentUser () {
56 |   try {
57 |     return JSON.parse(localStorage.getItem('user')) || 'null'
58 |   } catch {
59 |     return null
60 |   }
61 | }
62 |
63 | router.beforeEach((to) => {
64 |   const token = localStorage.getItem('token')
65 |   const user = currentUser()
66 |
67 |   if (to.meta.requiresAuth && !token) {
68 |     return { path: '/login', query: { redirect: to.fullPath } }
69 |   }
70 | })
```

```
70 |   if (to.meta.requiresAdmin && user?.role !== 'admin') {  
71 |     return { path: '/' }  
72 |   }  
73 |  
74 |   document.title = to.meta.title ? `${to.meta.title} · SmartLib` : 'SmartLib'  
75 | })  
76 |  
77 | export default router
```

9. Frontend: Axios client + JWT interceptor

Path: frontend/src/api/http.js Lines: 48

```
1 | /**
2 |  * Centralised Axios client for SmartLib.
3 |  *
4 |  * Import this in any component:
5 |  *   import http from '@api/http'
6 |  *   const { data } = await http.get('/api/books')
7 |  *
8 |  * - baseURL points at the Laragon-served Slim API
9 |  * - request interceptor adds the JWT from localStorage if present
10 |  * - response interceptor catches 401s and bounces the user to /login
11 |  */
12 |
13 | import axios from 'axios'
14 | import router from '@router'
15 | import { token, logout } from '@api/auth'
16 |
17 | const http = axios.create({
18 |   baseURL: import.meta.env.VITE_API_BASE_URL || 'http://smartlib.test',
19 |   timeout: 10_000,
20 |   headers: { 'Content-Type': 'application/json' }
21 | })
22 |
23 | // Attach the JWT (if any) to every outgoing request
24 | http.interceptors.request.use(config => {
25 |   if (token.value) {
26 |     config.headers.Authorization = `Bearer ${token.value}`
27 |   }
28 |   return config
29 | })
30 |
31 | // If the server says the token is gone/expired, send the user back to login
32 | http.interceptors.response.use(
33 |   res => res,
34 |   err => {
35 |     if (err.response?.status === 401) {
36 |       // Clears the reactive auth state AND localStorage in one step,
37 |       // so the navbar instantly reflects the logged-out state.
38 |       logout()
39 |       // Avoid redirect loops if we're already on /login
40 |       if (router.currentRoute.value.path !== '/login') {
41 |         router.push('/login')
42 |       }
43 |     }
44 |     return Promise.reject(err)
45 |   }
46 | )
47 |
48 | export default http
```

10. Frontend: Reactive auth state

Path: frontend/src/api/auth.js Lines: 56

```
1 | /**
2 |  * Shared auth state – a single reactive source of truth for the logged-in user.
3 |  *
4 |  * Why this exists:
5 |  * localStorage is not reactive. If you do `localStorage.setItem('user', ...)`
6 |  * during login, components reading from localStorage via `computed()` won't
7 |  * re-render until the user manually refreshes the page.
8 |  *
9 |  * This module wraps the user + token in Vue `ref`s so any component that
10 |  * imports `user` (or `isAdmin`) automatically re-renders on login/logout.
11 |  * localStorage is updated alongside the ref so the session survives reloads.
12 |  */
13 |
14 | import { ref, computed } from 'vue'
15 |
16 | // ■■■ Internal state – initialised from localStorage at module load ■■■
17 | function readUser () {
18 |   try {
19 |     return JSON.parse(localStorage.getItem('user') || 'null')
20 |   } catch {
21 |     return null
22 |   }
23 | }
24 |
25 | const _user = ref(readUser())
26 | const _token = ref(localStorage.getItem('token'))
27 |
28 | // ■■■ Public reactive getters ■■■
29 | export const user = computed(() => _user.value)
30 | export const token = computed(() => _token.value)
31 | export const isAuthenticated = computed(() => !!_token.value)
32 | export const isAdmin = computed(() => _user.value?.role === 'admin')
33 |
34 | // ■■■ Mutators ■■■
35 | /**
36 |  * Call this after a successful POST /api/members/login.
37 |  * @param {object} userData the `user` object returned by the server
38 |  * @param {string} tokenStr the JWT returned by the server
39 |  */
40 | export function login (userData, tokenStr) {
41 |   _user.value = userData
42 |   _token.value = tokenStr
43 |   localStorage.setItem('user', JSON.stringify(userData))
44 |   localStorage.setItem('token', tokenStr)
45 | }
46 |
47 | /**
48 |  * Clears local session. Does NOT call the backend – JWTs are stateless,
49 |  * so logout is purely a client-side concern.
50 |  */
51 | export function logout () {
52 |   _user.value = null
53 |   _token.value = null
54 |   localStorage.removeItem('user')
55 |   localStorage.removeItem('token')
56 | }
```

11. Frontend: Root component (App.vue)

Path: frontend/src/App.vue Lines: 140

```
1 | <template>
2 |   <div class="app-shell">
3 |     <!-- Top navigation -->
4 |     <header v-if="!$route.meta.hideNav" class="navbar">
5 |       <RouterLink to="/" class="brand">
6 |         <span class="brand-mark">■</span>
7 |         <span class="brand-name">SmartLib</span>
8 |       </RouterLink>
9 |
10 |       <nav class="nav-links">
11 |         <RouterLink to="/">Catalogue</RouterLink>
12 |         <RouterLink v-if="user" to="/my-borrows">My Borrows</RouterLink>
13 |         <RouterLink v-if="user?.role === 'admin'" to="/admin">Admin</RouterLink>
14 |       </nav>
15 |
16 |       <div class="nav-auth">
17 |         <template v-if="user">
18 |           <RouterLink to="/account" class="user-name" title="Change password">{{ user.name }}</RouterLink>
19 |           <button class="btn btn-ghost" @click="logout">Log out</button>
20 |         </template>
21 |         <template v-else>
22 |           <RouterLink to="/login" class="btn btn-ghost">Log in</RouterLink>
23 |           <RouterLink to="/register" class="btn btn-primary">Sign up</RouterLink>
24 |         </template>
25 |       </div>
26 |     </header>
27 |
28 |     <!-- Routed view -->
29 |     <main class="app-main">
30 |       <RouterView />
31 |     </main>
32 |
33 |     <!-- Footer -->
34 |     <footer v-if="!$route.meta.hideNav" class="app-footer">
35 |       <small>SmartLib · Polar Bear · SCSM2223</small>
36 |     </footer>
37 |   </div>
38 | </template>
39 |
40 | <script setup>
41 | import { useRouter, RouterLink, RouterView } from 'vue-router'
42 | import { user, logout as clearAuth } from '@api/auth'
43 |
44 | const router = useRouter()
45 |
46 | function logout () {
47 |   clearAuth()
48 |   router.push('/login')
49 | }
50 | </script>
51 |
52 | <style scoped>
53 | .app-shell {
54 |   min-height: 100vh;
55 |   display: flex;
56 |   flex-direction: column;
57 | }
58 |
59 | .navbar {
60 |   display: flex;
61 |   align-items: center;
62 |   gap: 1.5rem;
63 |   padding: 0.85rem 1.5rem;
64 |   background: var(--surface);
65 |   border-bottom: 1px solid var(--border);
66 |   box-shadow: 0 2px 4px rgba(0,0,0,0.03);
67 |   position: sticky;
68 |   top: 0;
```

```

69 |   z-index: 10;
70 | }
71 |
72 | .brand {
73 |   display: flex;
74 |   align-items: center;
75 |   gap: 0.5rem;
76 |   font-weight: 700;
77 |   font-size: 1.15rem;
78 |   text-decoration: none;
79 |   color: var(--text);
80 | }
81 | .brand-mark { font-size: 1.5rem; }
82 |
83 | .nav-links {
84 |   display: flex;
85 |   gap: 1.2rem;
86 |   flex: 1;
87 | }
88 | .nav-links a {
89 |   text-decoration: none;
90 |   color: var(--text-muted);
91 |   font-weight: 500;
92 |   padding: 0.4rem 0;
93 |   border-bottom: 2px solid transparent;
94 |   transition: color 0.15s, border-color 0.15s;
95 | }
96 | .nav-links a:hover { color: var(--text); }
97 | .nav-links a.router-link-exact-active {
98 |   color: var(--primary);
99 |   border-bottom-color: var(--primary);
100 | }
101 |
102 | .nav-auth {
103 |   display: flex;
104 |   align-items: center;
105 |   gap: 0.75rem;
106 | }
107 | .user-name {
108 |   color: var(--text-muted);
109 |   font-size: 0.9rem;
110 |   text-decoration: none;
111 |   padding: 0.3rem 0.5rem;
112 |   border-radius: var(--radius);
113 |   transition: background 0.15s, color 0.15s;
114 | }
115 | .user-name:hover {
116 |   background: var(--surface-muted);
117 |   color: var(--text);
118 | }
119 |
120 | .app-main {
121 |   flex: 1;
122 |   padding: 1.5rem;
123 |   max-width: 1200px;
124 |   width: 100%;
125 |   margin: 0 auto;
126 | }
127 |
128 | .app-footer {
129 |   text-align: center;
130 |   padding: 1rem;
131 |   color: var(--text-muted);
132 |   border-top: 1px solid var(--border);
133 | }
134 |
135 | @media (max-width: 640px) {
136 |   .navbar { flex-wrap: wrap; gap: 0.75rem; }
137 |   .nav-links { order: 3; width: 100%; justify-content: center; }
138 |   .app-main { padding: 1rem; }
139 | }
140 | </style>

```

12. Frontend: Book Catalogue view

Path: frontend/src/views/BookListView.vue Lines: 285

```
1 | <template>
2 |   <section class="book-list">
3 |     <header class="page-head">
4 |       <div>
5 |         <h1>Catalogue</h1>
6 |         <p class="muted">Browse books and borrow with one click.</p>
7 |       </div>
8 |       <SearchBar @search="handleSearch" />
9 |     </header>
10 |
11 |     <!-- Loading skeleton grid -->
12 |     <div v-if="isLoading" class="grid skeleton-grid">
13 |       <div v-for="n in 6" :key="n" class="skeleton-card"></div>
14 |     </div>
15 |
16 |     <!-- Error state -->
17 |     <div v-else-if="error" class="alert alert-error">
18 |       {{ error }}
19 |       <button class="btn btn-ghost" @click="loadBooks">Retry</button>
20 |     </div>
21 |
22 |     <!-- Empty state -->
23 |     <div v-else-if="filteredBooks.length === 0" class="empty">
24 |       <p class="muted">No books match your search.</p>
25 |     </div>
26 |
27 |     <!-- Real grid -->
28 |     <div v-else class="grid">
29 |       <BookCard
30 |         v-for="book in filteredBooks"
31 |         :key="book.id"
32 |         :book="book"
33 |         @borrow="handleBorrow"
34 |       />
35 |     </div>
36 |
37 |     <!-- Toast for borrow result -->
38 |     <transition name="toast-fade">
39 |       <div
40 |         v-if="toast"
41 |         class="toast alert"
42 |         :class="'alert-${toast.type}'"
43 |         role="status"
44 |       >{{ toast.msg }}</div>
45 |     </transition>
46 |
47 |     <!-- Confirm-borrow modal -->
48 |     <transition name="modal-fade">
49 |       <div
50 |         v-if="pendingBook"
51 |         class="modal-backdrop"
52 |         @click.self="cancelBorrow"
53 |       >
54 |         <div class="modal card" role="dialog" aria-modal="true">
55 |           <h2>Confirm borrow</h2>
56 |           <p class="muted">You're about to borrow:</p>
57 |           <div class="book-summary">
58 |             <strong>{{ pendingBook.title }}</strong>
59 |             <small class="muted">by {{ pendingBook.author }}</small>
60 |           </div>
61 |           <p class="muted hint">
62 |             Due in 14 days. You can return it any time from
63 |             <em>My Borrows</em>.
64 |           </p>
65 |           <div class="modal-actions">
66 |             <button
67 |               class="btn btn-ghost"
68 |               :disabled="borrowing"
69 |               @click="cancelBorrow"
```

```

70 |         >Cancel</button>
71 |     <button
72 |         class="btn btn-primary"
73 |         :disabled="borrowing"
74 |         @click="confirmBorrow"
75 |     >{{ borrowing ? 'Borrowing...' : 'Confirm borrow' }}</button>
76 | </div>
77 | </div>
78 | </div>
79 | </transition>
80 | </section>
81 | </template>
82 |
83 | <script setup>
84 | import { ref, computed, onMounted } from 'vue'
85 | import { useRouter } from 'vue-router'
86 | import http from '@/api/http'
87 | import BookCard from '@/components/BookCard.vue'
88 | import SearchBar from '@/components/SearchBar.vue'
89 |
90 | const router      = useRouter()
91 | const books       = ref([])
92 | const isLoading   = ref(true)
93 | const error       = ref('')
94 | const searchQuery = ref('')
95 | const toast       = ref(null)
96 | const pendingBook = ref(null) // book queued for borrow confirmation
97 | const borrowing   = ref(false) // true while POST /api/borrow is in flight
98 |
99 | // Client-side filter – instant feedback, no network call per keystroke
100 | const filteredBooks = computed(() => {
101 |     const q = searchQuery.value.toLowerCase()
102 |     if (!q) return books.value
103 |     return books.value.filter(b =>
104 |         b.title.toLowerCase().includes(q) ||
105 |         b.author.toLowerCase().includes(q) ||
106 |         (b.category || '').toLowerCase().includes(q)
107 |     )
108 | })
109 |
110 | function handleSearch (query) { searchQuery.value = query }
111 |
112 | // Step 1 – clicking Borrow on a card opens the confirmation modal.
113 | function handleBorrow (bookId) {
114 |     if (!localStorage.getItem('token')) {
115 |         router.push({ path: '/login', query: { redirect: '/' } })
116 |         return
117 |     }
118 |     const book = books.value.find(b => b.id === bookId)
119 |     if (book) pendingBook.value = book
120 | }
121 |
122 | // Step 2 – user clicked "Confirm borrow" in the modal: fire the API call.
123 | async function confirmBorrow () {
124 |     if (!pendingBook.value) return
125 |     borrowing.value = true
126 |     try {
127 |         await http.post('/api/borrow', { book_id: pendingBook.value.id })
128 |         // Optimistic UI: decrement available_count for instant feedback
129 |         pendingBook.value.available_count = Math.max(0, pendingBook.value.available_count - 1)
130 |         showToast('success', `Borrowed "${pendingBook.value.title}". Check "My Borrows" for the due date.`)
131 |         pendingBook.value = null
132 |     } catch (err) {
133 |         showToast('error', err.response?.data?.message ?? 'Borrow failed.')
134 |         pendingBook.value = null
135 |     } finally {
136 |         borrowing.value = false
137 |     }
138 | }
139 |
140 | function cancelBorrow () {
141 |     if (borrowing.value) return // ignore clicks while request is mid-flight

```

```

142 |   pendingBook.value = null
143 | }
144 |
145 | function showToast (type, msg) {
146 |   toast.value = { type, msg }
147 |   setTimeout(() => { toast.value = null }, 3500)
148 | }
149 |
150 | async function loadBooks () {
151 |   isLoading.value = true
152 |   error.value = ''
153 |   try {
154 |     const { data } = await http.get('/api/books')
155 |     books.value = data.data || []
156 |   } catch (err) {
157 |     error.value = err.response?.data?.message ?? 'Failed to load books.'
158 |   } finally {
159 |     isLoading.value = false
160 |   }
161 | }
162 |
163 | onMounted(loadBooks)
164 | </script>
165 |
166 | <style scoped>
167 | .page-head {
168 |   display: flex;
169 |   flex-direction: column;
170 |   gap: 1rem;
171 |   margin-bottom: 1.5rem;
172 | }
173 | @media (min-width: 768px) {
174 |   .page-head {
175 |     flex-direction: row;
176 |     align-items: flex-end;
177 |     justify-content: space-between;
178 |   }
179 |   .page-head > :last-child { min-width: 320px; }
180 | }
181 |
182 | .grid {
183 |   display: grid;
184 |   grid-template-columns: repeat(3, 1fr);
185 |   gap: 1.25rem;
186 | }
187 | @media (max-width: 1024px) {
188 |   .grid { grid-template-columns: repeat(2, 1fr); }
189 | }
190 | @media (max-width: 600px) {
191 |   .grid { grid-template-columns: 1fr; }
192 | }
193 |
194 | /* Shimmer skeleton */
195 | .skeleton-card {
196 |   height: 280px;
197 |   border-radius: var(--radius-lg);
198 |   background: linear-gradient(90deg, #f0f4f8 25%, #e2ecf4 50%, #f0f4f8 75%);
199 |   background-size: 200% 100%;
200 |   animation: shimmer 1.5s infinite;
201 | }
202 | @keyframes shimmer {
203 |   0% { background-position: 200% 0; }
204 |   100% { background-position: -200% 0; }
205 | }
206 |
207 | .empty {
208 |   text-align: center;
209 |   padding: 4rem 1rem;
210 | }
211 |
212 | /* Floating toast – always visible regardless of scroll */
213 | .toast {
214 |   position: fixed;

```

```

215 | top: 80px;          /* sits just below the navbar */
216 | left: 50%;
217 | transform: translateX(-50%);
218 | z-index: 100;
219 | min-width: 280px;
220 | max-width: 90vw;
221 | margin: 0;
222 | text-align: center;
223 | box-shadow: var(--shadow);
224 | }
225 | .toast-fade-enter-active,
226 | .toast-fade-leave-active {
227 |   transition: opacity 0.25s ease, transform 0.25s ease;
228 | }
229 | .toast-fade-enter-from,
230 | .toast-fade-leave-to {
231 |   opacity: 0;
232 |   transform: translate(-50%, -8px);
233 | }
234 |
235 | /* Confirm-borrow modal */
236 | .modal-backdrop {
237 |   position: fixed;
238 |   inset: 0;
239 |   background: rgba(15, 23, 42, 0.45);
240 |   display: flex;
241 |   align-items: center;
242 |   justify-content: center;
243 |   padding: 1rem;
244 |   z-index: 50;
245 | }
246 | .modal {
247 |   width: 100%;
248 |   max-width: 420px;
249 |   display: flex;
250 |   flex-direction: column;
251 |   gap: 0.75rem;
252 | }
253 | .book-summary {
254 |   background: var(--surface-muted);
255 |   padding: 0.75rem 1rem;
256 |   border-radius: var(--radius);
257 |   display: flex;
258 |   flex-direction: column;
259 |   gap: 0.15rem;
260 | }
261 | .book-summary strong { font-size: 1rem; color: var(--text); }
262 | .hint { font-size: 0.85rem; }
263 | .modal-actions {
264 |   display: flex;
265 |   justify-content: flex-end;
266 |   gap: 0.5rem;
267 |   margin-top: 0.5rem;
268 | }
269 | .modal-fade-enter-active,
270 | .modal-fade-leave-active {
271 |   transition: opacity 0.2s ease;
272 | }
273 | .modal-fade-enter-active .modal,
274 | .modal-fade-leave-active .modal {
275 |   transition: transform 0.2s ease;
276 | }
277 | .modal-fade-enter-from,
278 | .modal-fade-leave-to {
279 |   opacity: 0;
280 | }
281 | .modal-fade-enter-from .modal,
282 | .modal-fade-leave-to .modal {
283 |   transform: translateY(-8px);
284 | }
285 | </style>

```

13. Frontend: Borrow view

Path: frontend/src/views/BorrowView.vue Lines: 74

```
1 | <template>
2 |   <section class="borrow-view">
3 |     <RouterLink to="/" class="back-link"><- Back to catalogue</RouterLink>
4 |
5 |     <LoadingSpinner v-if="loading" label="Loading book..." />
6 |
7 |     <div v-else-if="error" class="alert alert-error">{{ error }}</div>
8 |
9 |     <div v-else-if="success" class="card stack">
10 |       <h2>You're all set!</h2>
11 |       <p>You've borrowed <strong>{{ book.title }}</strong>.</p>
12 |       <p class="muted">Due date: <strong>{{ success.due_date }}</strong></p>
13 |       <div class="flex">
14 |         <RouterLink to="/my-borrows" class="btn btn-primary">View my borrows</RouterLink>
15 |         <RouterLink to="/" class="btn btn-ghost">Browse more</RouterLink>
16 |       </div>
17 |     </div>
18 |
19 |     <BorrowForm
20 |       v-else-if="book"
21 |       :book="book"
22 |       @borrowed="onBorrowed"
23 |       @cancel="router.push('/')"
24 |     />
25 |   </section>
26 | </template>
27 |
28 | <script setup>
29 | import { ref, onMounted } from 'vue'
30 | import { useRoute, useRouter, RouterLink } from 'vue-router'
31 | import http from '@api/http'
32 | import BorrowForm from '@components/BorrowForm.vue'
33 | import LoadingSpinner from '@components/LoadingSpinner.vue'
34 |
35 | const route = useRoute()
36 | const router = useRouter()
37 | const book = ref(null)
38 | const loading = ref(true)
39 | const error = ref('')
40 | const success = ref(null)
41 |
42 | async function loadBook () {
43 |   loading.value = true
44 |   try {
45 |     const { data } = await http.get(`/api/books/${route.params.id}`)
46 |     book.value = data.data
47 |   } catch (err) {
48 |     error.value = err.response?.data?.message ?? 'Could not load book.'
49 |   } finally {
50 |     loading.value = false
51 |   }
52 | }
53 |
54 | function onBorrowed (payload) {
55 |   success.value = payload
56 | }
57 |
58 | onMounted(loadBook)
59 | </script>
60 |
61 | <style scoped>
62 | .borrow-view {
63 |   max-width: 560px;
64 |   margin: 0 auto;
65 | }
66 | .back-link {
67 |   display: inline-block;
68 |   margin-bottom: 1rem;
69 |   color: var(--text-muted);
```

```
70 |     text-decoration: none;
71 |     font-size: 0.9rem;
72 | }
73 | .back-link:hover { color: var(--text); }
74 | </style>
```

14. Frontend: My Borrows / Return view

Path: frontend/src/views/ReturnView.vue Lines: 151

```
1 | <template>
2 |   <section class="return-view">
3 |     <header class="page-head">
4 |       <h1>My Borrows</h1>
5 |       <p class="muted">All books you've borrowed, current and past.</p>
6 |     </header>
7 |
8 |     <LoadingSpinner v-if="loading" label="Loading your borrows..." />
9 |
10 |    <div v-else-if="error" class="alert alert-error">{{ error }}</div>
11 |
12 |    <div v-else-if="records.length === 0" class="empty card">
13 |      <p class="muted">You haven't borrowed any books yet.</p>
14 |      <RouterLink to="/" class="btn btn-primary">Browse catalogue</RouterLink>
15 |    </div>
16 |
17 |    <div v-else class="stack">
18 |      <div v-if="toast" class="alert" :class="'alert-${toast.type}'">{{ toast.msg }}</div>
19 |
20 |      <div class="table-wrap">
21 |        <table class="table">
22 |          <thead>
23 |            <tr>
24 |              <th>Book</th>
25 |              <th>Borrowed</th>
26 |              <th>Due</th>
27 |              <th>Status</th>
28 |            </tr>
29 |          </thead>
30 |          <tbody>
31 |            <tr v-for="r in records" :key="r.id">
32 |              <td>
33 |                <div class="book-cell">
34 |                  <strong>{{ r.title }}</strong>
35 |                  <small class="muted">by {{ r.author }}</small>
36 |                </div>
37 |              </td>
38 |              <td>{{ r.borrow_date }}</td>
39 |              <td>{{ r.due_date }}</td>
40 |              <td>
41 |                <span class="badge" :class="badgeClass(r)">
42 |                  {{ statusLabel(r) }}
43 |                </span>
44 |              </td>
45 |              <td>
46 |                <button
47 |                  v-if="r.status === 'active'"
48 |                  class="btn btn-primary"
49 |                  :disabled="returning === r.id"
50 |                  @click="returnBook(r.id)"
51 |                >
52 |                  {{ returning === r.id ? 'Returning...' : 'Return' }}
53 |                </button>
54 |                <span v-else class="muted">—</span>
55 |              </td>
56 |            </tr>
57 |          </tbody>
58 |        </table>
59 |      </div>
60 |    </div>
61 |  </section>
62 | </template>
63 |
64 |
65 | <script setup>
66 | import { ref, onMounted } from 'vue'
67 | import { RouterLink } from 'vue-router'
68 | import http from '@api/http'
69 | import LoadingSpinner from '@components/LoadingSpinner.vue'
```

```

70 |
71 | const records    = ref([])
72 | const loading    = ref(true)
73 | const error      = ref('')
74 | const returning  = ref(null)
75 | const toast      = ref(null)
76 |
77 | async function loadBorrows () {
78 |   loading.value = true
79 |   try {
80 |     const { data } = await http.get('/api/borrow/me')
81 |     records.value = data.data || []
82 |   } catch (err) {
83 |     error.value = err.response?.data?.message ?? 'Failed to load borrows.'
84 |   } finally {
85 |     loading.value = false
86 |   }
87 | }
88 |
89 | async function returnBook (recordId) {
90 |   returning.value = recordId
91 |   try {
92 |     await http.post(`/api/return/${recordId}`)
93 |     showToast('success', 'Book returned. Thanks!')
94 |     await loadBorrows()
95 |   } catch (err) {
96 |     showToast('error', err.response?.data?.message ?? 'Return failed.')
97 |   } finally {
98 |     returning.value = null
99 |   }
100 | }
101 |
102 | function statusLabel (r) {
103 |   if (r.status === 'returned') return 'Returned'
104 |   if (r.status === 'overdue' || (r.status === 'active' && new Date(r.due_date) < new Date())) {
105 |     return 'Overdue'
106 |   }
107 |   return 'Active'
108 | }
109 |
110 | function badgeClass (r) {
111 |   const label = statusLabel(r)
112 |   if (label === 'Returned') return 'badge-success'
113 |   if (label === 'Overdue') return 'badge-danger'
114 |   return ''
115 | }
116 |
117 | function showToast (type, msg) {
118 |   toast.value = { type, msg }
119 |   setTimeout(() => { toast.value = null }, 3500)
120 | }
121 |
122 | onMounted(loadBorrows)
123 | </script>
124 |
125 | <style scoped>
126 | .page-head { margin-bottom: 1.5rem; }
127 | .table-wrap { overflow-x: auto; }
128 | .book-cell {
129 |   display: flex;
130 |   flex-direction: column;
131 | }
132 | .badge {
133 |   background: var(--surface-muted);
134 |   color: var(--text-muted);
135 |   padding: 0.2rem 0.55rem;
136 |   border-radius: 999px;
137 |   font-size: 0.72rem;
138 |   font-weight: 500;
139 | }
140 | .badge-success { background: var(--success-soft); color: var(--success); }
141 | .badge-danger { background: var(--danger-soft); color: var(--danger); }
142 |

```

```
143 | .empty {  
144 |   text-align: center;  
145 |   padding: 3rem 1rem;  
146 |   display: flex;  
147 |   flex-direction: column;  
148 |   align-items: center;  
149 |   gap: 1rem;  
150 | }  
151 | </style>
```

15. Frontend: Admin CRUD view

Path: frontend/src/views/AdminView.vue Lines: 206

```
1 | <template>
2 |   <section class="admin-view">
3 |     <header class="page-head">
4 |       <div>
5 |         <h1>Admin Dashboard</h1>
6 |         <p class="muted">Manage the catalogue. {{ books.length }} title(s).</p>
7 |       </div>
8 |       <button class="btn btn-primary" @click="openCreate">+ Add book</button>
9 |     </header>
10 |
11 |     <LoadingSpinner v-if="loading" label="Loading..." />
12 |     <div v-else-if="error" class="alert alert-error">{{ error }}</div>
13 |
14 |     <div v-else class="stack">
15 |       <div v-if="toast" class="alert" :class="'alert-${toast.type}'">{{ toast.msg }}</div>
16 |
17 |       <div class="table-wrap">
18 |         <table class="table">
19 |           <thead>
20 |             <tr>
21 |               <th>Title</th>
22 |               <th>Author</th>
23 |               <th>Category</th>
24 |               <th>Stock</th>
25 |               <th>Available</th>
26 |             </tr>
27 |           </thead>
28 |           <tbody>
29 |             <tr v-for="b in books" :key="b.id">
30 |               <td>{{ b.title }}</td>
31 |               <td>{{ b.author }}</td>
32 |               <td>{{ b.category || '-' }}</td>
33 |               <td>{{ b.stock }}</td>
34 |               <td>{{ b.available_count }}</td>
35 |               <td class="row-actions">
36 |                 <button class="btn btn-ghost" @click="openEdit(b)">Edit</button>
37 |                 <button class="btn btn-danger" @click="confirmDelete(b)">Delete</button>
38 |               </td>
39 |             </tr>
40 |           </tbody>
41 |         </table>
42 |       </div>
43 |     </div>
44 |   </div>
45 |
46 |   <!-- Modal: create / edit -->
47 |   <div v-if="modal.open" class="modal-backdrop" @click.self="closeModal">
48 |     <form class="modal card stack" @submit.prevent="saveBook">
49 |       <h2>{{ modal.mode === 'edit' ? 'Edit book' : 'Add book' }}</h2>
50 |
51 |       <div class="form-group">
52 |         <label>Title</label>
53 |         <input v-model="modal.form.title" class="form-input" required />
54 |       </div>
55 |       <div class="form-group">
56 |         <label>Author</label>
57 |         <input v-model="modal.form.author" class="form-input" required />
58 |       </div>
59 |       <div class="form-group">
60 |         <label>ISBN (13 digits)</label>
61 |         <input v-model="modal.form.isbn" class="form-input" maxlength="13" pattern="\d{13}" />
62 |       </div>
63 |       <div class="form-group">
64 |         <label>Category</label>
65 |         <input v-model="modal.form.category" class="form-input" />
66 |       </div>
67 |       <div class="form-group">
68 |         <label>Stock</label>
69 |         <input v-model="modal.form.stock" type="number" min="1" class="form-input" require
```

```

    d />
70 |         </div>
71 |
72 |         <div v-if="modal.error" class="alert alert-error">{{ modal.error }}</div>
73 |
74 |         <div class="flex between">
75 |             <button type="button" class="btn btn-ghost" @click="closeModal">Cancel</button>
76 |             <button type="submit" class="btn btn-primary" :disabled="modal.saving">
77 |                 {{ modal.saving ? 'Saving...' : 'Save' }}
78 |             </button>
79 |         </div>
80 |     </form>
81 | </div>
82 | </section>
83 | </template>
84 |
85 | <script setup>
86 | import { ref, reactive, onMounted } from 'vue'
87 | import http from '@api/http'
88 | import LoadingSpinner from '@components/LoadingSpinner.vue'
89 |
90 | const books = ref([])
91 | const loading = ref(true)
92 | const error = ref('')
93 | const toast = ref(null)
94 |
95 | const blankForm = () => ({
96 |   id: null, title: '', author: '', isbn: '', category: '', stock: 1
97 | })
98 |
99 | const modal = reactive({
100 |   open: false,
101 |   mode: 'create', // 'create' | 'edit'
102 |   form: blankForm(),
103 |   saving: false,
104 |   error: ''
105 | })
106 |
107 | async function loadBooks () {
108 |   loading.value = true
109 |   try {
110 |     const { data } = await http.get('/api/books')
111 |     books.value = data.data || []
112 |   } catch (err) {
113 |     error.value = err.response?.data?.message ?? 'Failed to load books.'
114 |   } finally {
115 |     loading.value = false
116 |   }
117 | }
118 |
119 | function openCreate () {
120 |   modal.mode = 'create'
121 |   modal.form = blankForm()
122 |   modal.error = ''
123 |   modal.open = true
124 | }
125 | function openEdit (book) {
126 |   modal.mode = 'edit'
127 |   modal.form = { ...book, isbn: book.isbn || '', category: book.category || '' }
128 |   modal.error = ''
129 |   modal.open = true
130 | }
131 | function closeModal () {
132 |   modal.open = false
133 | }
134 |
135 | async function saveBook () {
136 |   modal.saving = true
137 |   modal.error = ''
138 |   try {
139 |     if (modal.mode === 'create') {
140 |       await http.post('/api/books', modal.form)
141 |       showToast('success', 'Book added.')

```

```

142 |     } else {
143 |         await http.put(`/api/books/${modal.form.id}`, modal.form)
144 |         showToast('success', 'Book updated.')
145 |     }
146 |     modal.open = false
147 |     await loadBooks()
148 | } catch (err) {
149 |     modal.error = err.response?.data?.message ?? 'Save failed.'
150 | } finally {
151 |     modal.saving = false
152 | }
153 | }
154 |
155 | async function confirmDelete (book) {
156 |     if (!confirm(`Delete "${book.title}"? This cannot be undone.`)) return
157 |     try {
158 |         await http.delete(`/api/books/${book.id}`)
159 |         showToast('success', 'Book deleted.')
160 |         await loadBooks()
161 |     } catch (err) {
162 |         showToast('error', err.response?.data?.message ?? 'Delete failed.')
163 |     }
164 | }
165 |
166 | function showToast (type, msg) {
167 |     toast.value = { type, msg }
168 |     setTimeout(() => { toast.value = null }, 3500)
169 | }
170 |
171 | onMounted(loadBooks)
172 | </script>
173 |
174 | <style scoped>
175 | .page-head {
176 |     display: flex;
177 |     justify-content: space-between;
178 |     align-items: flex-end;
179 |     flex-wrap: wrap;
180 |     gap: 1rem;
181 |     margin-bottom: 1.5rem;
182 | }
183 | .table-wrap { overflow-x: auto; }
184 | .row-actions {
185 |     display: flex;
186 |     gap: 0.4rem;
187 |     justify-content: flex-end;
188 | }
189 |
190 | .modal-backdrop {
191 |     position: fixed;
192 |     inset: 0;
193 |     background: rgba(15, 23, 42, 0.45);
194 |     display: flex;
195 |     align-items: center;
196 |     justify-content: center;
197 |     padding: 1rem;
198 |     z-index: 50;
199 | }
200 | .modal {
201 |     width: 100%;
202 |     max-width: 480px;
203 |     max-height: 90vh;
204 |     overflow-y: auto;
205 | }
206 | </style>

```

16. Frontend: Login view

Path: frontend/src/views/LoginView.vue Lines: 115

```
1 | <template>
2 |   <section class="auth-page">
3 |     <div class="auth-card card">
4 |       <div class="brand-row">
5 |         <span class="brand-mark">■</span>
6 |         <h1>Welcome back</h1>
7 |       </div>
8 |       <p class="muted">Log in to borrow books and view your history.</p>
9 |
10 |      <form @submit.prevent="login">
11 |        <div class="form-group">
12 |          <label for="email">Email</label>
13 |          <input
14 |            id="email"
15 |            v-model="email"
16 |            type="email"
17 |            class="form-input"
18 |            autocomplete="email"
19 |            required
20 |          />
21 |        </div>
22 |
23 |        <div class="form-group">
24 |          <label for="password">Password</label>
25 |          <input
26 |            id="password"
27 |            v-model="password"
28 |            type="password"
29 |            class="form-input"
30 |            autocomplete="current-password"
31 |            required
32 |          />
33 |        </div>
34 |
35 |        <div v-if="error" class="alert alert-error">{{ error }}</div>
36 |
37 |        <button
38 |          type="submit"
39 |          class="btn btn-primary"
40 |          style="width:100%"
41 |          :disabled="loading"
42 |        >
43 |          {{ loading ? 'Logging in...' : 'Log in' }}
44 |        </button>
45 |      </form>
46 |
47 |      <p class="register-link">
48 |        Don't have an account?
49 |        <RouterLink to="/register">Create one</RouterLink>
50 |      </p>
51 |    </div>
52 |  </section>
53 | </template>
54 |
55 | <script setup>
56 | import { ref } from 'vue'
57 | import { useRouter, useRoute, RouterLink } from 'vue-router'
58 | import http from '@/api/http'
59 | import { login as setAuth } from '@/api/auth'
60 |
61 | const router = useRouter()
62 | const route = useRoute()
63 |
64 | const email = ref('')
65 | const password = ref('')
66 | const error = ref('')
67 | const loading = ref(false)
68 |
69 | async function login () {
```

```

70 | error.value = ''
71 | loading.value = true
72 | try {
73 |   const { data } = await http.post('/api/members/login', {
74 |     email: email.value,
75 |     password: password.value
76 |   })
77 |   // Use the shared auth module – updates the reactive ref AND localStorage
78 |   // in one step, so the navbar updates instantly without a page reload.
79 |   setAuth(data.user, data.token)
80 |   router.push(route.query.redirect || '/')
81 | } catch (err) {
82 |   error.value = err.response?.data?.message ?? 'Login failed. Please try again.'
83 | } finally {
84 |   loading.value = false
85 | }
86 | }
87 | </script>
88 |
89 | <style scoped>
90 | .auth-page {
91 |   display: flex;
92 |   align-items: center;
93 |   justify-content: center;
94 |   min-height: 80vh;
95 | }
96 | .auth-card {
97 |   width: 100%;
98 |   max-width: 420px;
99 | }
100 | .brand-row {
101 |   display: flex;
102 |   align-items: center;
103 |   gap: 0.5rem;
104 |   margin-bottom: 0.5rem;
105 | }
106 | .brand-mark { font-size: 1.75rem; }
107 | .register-link {
108 |   text-align: center;
109 |   margin-top: 1rem;
110 |   font-size: 0.9rem;
111 |   color: var(--text-muted);
112 | }
113 | .register-link a { color: var(--primary); text-decoration: none; }
114 | .register-link a:hover { text-decoration: underline; }
115 | </style>

```

17. Frontend: Register view

Path: frontend/src/views/RegisterView.vue Lines: 91

```
1 | <template>
2 |   <section class="auth-page">
3 |     <div class="auth-card card">
4 |       <h1>Create an account</h1>
5 |       <p class="muted">Sign up to start borrowing books.</p>
6 |
7 |       <form @submit.prevent="register">
8 |         <div class="form-group">
9 |           <label for="name">Full name</label>
10 |          <input id="name" v-model="form.name" type="text" class="form-input" required />
11 |        </div>
12 |
13 |        <div class="form-group">
14 |          <label for="email">Email</label>
15 |          <input id="email" v-model="form.email" type="email" class="form-input" required />
16 |        </div>
17 |
18 |        <div class="form-group">
19 |          <label for="password">Password</label>
20 |          <input
21 |            id="password"
22 |            v-model="form.password"
23 |            type="password"
24 |            class="form-input"
25 |            minlength="8"
26 |            required
27 |          />
28 |          <p class="muted hint">At least 8 characters.</p>
29 |        </div>
30 |
31 |        <div v-if="error" class="alert alert-error">{{ error }}</div>
32 |        <div v-if="success" class="alert alert-success">{{ success }}</div>
33 |
34 |        <button type="submit" class="btn btn-primary" style="width:100%" :disabled="loading">
35 |          {{ loading ? 'Creating account...' : 'Sign up' }}
36 |        </button>
37 |      </form>
38 |
39 |      <p class="register-link">
40 |        Already have an account?
41 |        <RouterLink to="/login">Log in</RouterLink>
42 |      </p>
43 |    </div>
44 |  </section>
45 | </template>
46 |
47 | <script setup>
48 | import { reactive, ref } from 'vue'
49 | import { useRouter, RouterLink } from 'vue-router'
50 | import http from '@/api/http'
51 |
52 | const router = useRouter()
53 | const form = reactive({ name: '', email: '', password: '' })
54 | const error = ref('')
55 | const success = ref('')
56 | const loading = ref(false)
57 |
58 | async function register () {
59 |   error.value = ''
60 |   success.value = ''
61 |   loading.value = true
62 |   try {
63 |     await http.post('/api/members/register', form)
64 |     success.value = 'Account created. Redirecting to login...'
65 |     setTimeout(() => router.push('/login'), 1200)
66 |   } catch (err) {
67 |     error.value = err.response?.data?.message ?? 'Registration failed.'
68 |   } finally {
69 |     loading.value = false
```

```
70 |   }
71 | }
72 | </script>
73 |
74 | <style scoped>
75 | .auth-page {
76 |   display: flex;
77 |   align-items: center;
78 |   justify-content: center;
79 |   min-height: 80vh;
80 | }
81 | .auth-card { width: 100%; max-width: 420px; }
82 | .hint { font-size: 0.8rem; margin-top: 0.25rem; }
83 | .register-link {
84 |   text-align: center;
85 |   margin-top: 1rem;
86 |   font-size: 0.9rem;
87 |   color: var(--text-muted);
88 | }
89 | .register-link a { color: var(--primary); text-decoration: none; }
90 | .register-link a:hover { text-decoration: underline; }
91 | </style>
```

18. Frontend: Change Password view

Path: frontend/src/views/ChangePasswordView.vue Lines: 138

```
1 | <template>
2 |   <section class="account-page">
3 |     <div class="account-card card">
4 |       <h1>Change password</h1>
5 |       <p class="muted">Update the password you use to log in.</p>
6 |
7 |       <form @submit.prevent="submit">
8 |         <div class="form-group">
9 |           <label for="current">Current password</label>
10 |           <input
11 |             id="current"
12 |             v-model="form.current"
13 |             type="password"
14 |             class="form-input"
15 |             autocomplete="current-password"
16 |             required
17 |           />
18 |         </div>
19 |
20 |         <div class="form-group">
21 |           <label for="new">New password</label>
22 |           <input
23 |             id="new"
24 |             v-model="form.next"
25 |             type="password"
26 |             class="form-input"
27 |             autocomplete="new-password"
28 |             minlength="8"
29 |             required
30 |           />
31 |           <p class="muted hint">At least 8 characters.</p>
32 |         </div>
33 |
34 |         <div class="form-group">
35 |           <label for="confirm">Confirm new password</label>
36 |           <input
37 |             id="confirm"
38 |             v-model="form.confirm"
39 |             type="password"
40 |             class="form-input"
41 |             autocomplete="new-password"
42 |             minlength="8"
43 |             required
44 |           />
45 |           <p
46 |             v-if="form.confirm && !confirmsMatch"
47 |             class="muted hint mismatch"
48 |             >Passwords don't match.</p>
49 |         </div>
50 |
51 |         <div v-if="error" class="alert alert-error">{{ error }}</div>
52 |         <div v-if="success" class="alert alert-success">{{ success }}</div>
53 |
54 |         <button
55 |           type="submit"
56 |           class="btn btn-primary"
57 |           style="width:100%"
58 |           :disabled="loading || !confirmsMatch"
59 |         >
60 |           {{ loading ? 'Updating...' : 'Update password' }}
61 |         </button>
62 |       </form>
63 |
64 |       <RouterLink to="/" class="back-link">← Back to catalogue</RouterLink>
65 |     </div>
66 |   </section>
67 | </template>
68 |
69 | <script setup>
```

```

70 | import { reactive, ref, computed } from 'vue'
71 | import { RouterLink } from 'vue-router'
72 | import http from '@api/http'
73 |
74 | const form = reactive({ current: '', next: '', confirm: '' })
75 | const error = ref('')
76 | const success = ref('')
77 | const loading = ref(false)
78 |
79 | // Client-side guard – backend also rejects mismatched passwords (defence in depth)
80 | const confirmsMatch = computed(() =>
81 |   form.confirm === '' || form.confirm === form.next
82 | )
83 |
84 | async function submit () {
85 |   error.value = ''
86 |   success.value = ''
87 |
88 |   if (form.next !== form.confirm) {
89 |     error.value = "New password and confirmation don't match."
90 |     return
91 |   }
92 |   if (form.next === form.current) {
93 |     error.value = 'New password must be different from the current one.'
94 |     return
95 |   }
96 |
97 |   loading.value = true
98 |   try {
99 |     await http.post('/api/members/password', {
100 |       current_password: form.current,
101 |       new_password: form.next
102 |     })
103 |     success.value = 'Password updated. You can keep using your current session.'
104 |     form.current = ''
105 |     form.next = ''
106 |     form.confirm = ''
107 |   } catch (err) {
108 |     error.value = err.response?.data?.message ?? 'Failed to update password.'
109 |   } finally {
110 |     loading.value = false
111 |   }
112 | }
113 | </script>
114 |
115 | <style scoped>
116 | .account-page {
117 |   display: flex;
118 |   align-items: center;
119 |   justify-content: center;
120 |   min-height: 70vh;
121 | }
122 | .account-card {
123 |   width: 100%;
124 |   max-width: 420px;
125 | }
126 | .hint { font-size: 0.8rem; margin-top: 0.25rem; }
127 | .mismatch { color: var(--danger); }
128 | .back-link {
129 |   display: inline-block;
130 |   margin-top: 1rem;
131 |   text-align: center;
132 |   width: 100%;
133 |   font-size: 0.9rem;
134 |   color: var(--text-muted);
135 |   text-decoration: none;
136 | }
137 | .back-link:hover { color: var(--text); }
138 | </style>

```

19. Frontend: Book Card component

Path: frontend/src/components/BookCard.vue Lines: 129

```
1 | <template>
2 |   <article class="book-card">
3 |     <div class="cover">
4 |       <span class="cover-icon">■</span>
5 |     </div>
6 |
7 |     <div class="body">
8 |       <h3 class="title">{{ book.title }}</h3>
9 |       <p class="author">by {{ book.author }}</p>
10 |
11 |       <div class="meta">
12 |         <span v-if="book.category" class="badge">{{ book.category }}</span>
13 |         <span
14 |           class="badge"
15 |           :class="available ? 'badge-success' : 'badge-danger'"
16 |         >
17 |           {{ available ? `${book.available_count} available` : 'Out of stock' }}
18 |         </span>
19 |       </div>
20 |
21 |       <div class="actions">
22 |         <button
23 |           class="btn btn-primary"
24 |           :disabled="!available"
25 |           @click="$emit('borrow', book.id)"
26 |         >
27 |           {{ available ? 'Borrow' : 'Unavailable' }}
28 |         </button>
29 |         <button
30 |           v-if="showAdmin"
31 |           class="btn btn-ghost"
32 |           @click="$emit('edit', book)"
33 |         >Edit</button>
34 |         <button
35 |           v-if="showAdmin"
36 |           class="btn btn-danger"
37 |           @click="$emit('delete', book.id)"
38 |         >Delete</button>
39 |       </div>
40 |     </div>
41 |   </article>
42 | </template>
43 |
44 | <script setup>
45 | import { computed } from 'vue'
46 |
47 | const props = defineProps({
48 |   book: { type: Object, required: true },
49 |   showAdmin: { type: Boolean, default: false }
50 | })
51 |
52 | defineEmits(['borrow', 'edit', 'delete'])
53 |
54 | const available = computed(() => Number(props.book.available_count) > 0)
55 | </script>
56 |
57 | <style scoped>
58 | .book-card {
59 |   display: flex;
60 |   flex-direction: column;
61 |   background: var(--surface);
62 |   border: 1px solid var(--border);
63 |   border-radius: var(--radius-lg);
64 |   overflow: hidden;
65 |   box-shadow: var(--shadow-sm);
66 |   transition: transform 0.15s, box-shadow 0.15s;
67 | }
68 | .book-card:hover {
69 |   transform: translateY(-2px);
```

```

70 |     box-shadow: var(--shadow);
71 | }
72 |
73 | .cover {
74 |     height: 120px;
75 |     background: linear-gradient(135deg, #dbeafe, #c7d2fe);
76 |     display: flex;
77 |     align-items: center;
78 |     justify-content: center;
79 | }
80 | .cover-icon { font-size: 3rem; }
81 |
82 | .body {
83 |     padding: 1rem;
84 |     display: flex;
85 |     flex-direction: column;
86 |     gap: 0.5rem;
87 |     flex: 1;
88 | }
89 |
90 | .title {
91 |     font-size: 1rem;
92 |     margin: 0;
93 |     /* clamp long titles to 2 lines */
94 |     display: -webkit-box;
95 |     -webkit-line-clamp: 2;
96 |     -webkit-box-orient: vertical;
97 |     overflow: hidden;
98 | }
99 | .author {
100 |     margin: 0;
101 |     font-size: 0.85rem;
102 |     color: var(--text-muted);
103 | }
104 |
105 | .meta {
106 |     display: flex;
107 |     flex-wrap: wrap;
108 |     gap: 0.4rem;
109 |     margin-top: auto;
110 | }
111 | .badge {
112 |     background: var(--surface-muted);
113 |     color: var(--text-muted);
114 |     padding: 0.2rem 0.55rem;
115 |     border-radius: 999px;
116 |     font-size: 0.72rem;
117 |     font-weight: 500;
118 | }
119 | .badge-success { background: var(--success-soft); color: var(--success); }
120 | .badge-danger { background: var(--danger-soft); color: var(--danger); }
121 |
122 | .actions {
123 |     display: flex;
124 |     flex-wrap: wrap;
125 |     gap: 0.4rem;
126 |     margin-top: 0.5rem;
127 | }
128 | .actions .btn { flex: 1; min-width: 0; }
129 | </style>

```

20. Frontend: Borrow Form component

Path: frontend/src/components/BorrowForm.vue Lines: 74

```
1 | <template>
2 |   <form class="card stack" @submit.prevent="submit">
3 |     <h2>Borrow "{{ book.title }}"</h2>
4 |     <p class="muted">By {{ book.author }} · {{ book.available_count }} copy(ies) available</p>
5 |
6 |     <div class="form-group">
7 |       <label for="duration">Borrow duration</label>
8 |       <select id="duration" v-model="durationDays" class="form-input">
9 |         <option :value="7">7 days</option>
10 |         <option :value="14">14 days (default)</option>
11 |         <option :value="21">21 days</option>
12 |       </select>
13 |       <p class="muted hint">
14 |         The backend enforces a fixed 14-day default – this field is informational.
15 |       </p>
16 |     </div>
17 |
18 |     <label class="terms-row">
19 |       <input v-model="agreed" type="checkbox" />
20 |       I agree to return the book by the due date.
21 |     </label>
22 |
23 |     <div v-if="error" class="alert alert-error">{{ error }}</div>
24 |
25 |     <div class="flex between">
26 |       <button type="button" class="btn btn-ghost" @click="$emit('cancel')">Cancel</button>
27 |       <button
28 |         type="submit"
29 |         class="btn btn-primary"
30 |         :disabled="!agreed || submitting || book.available_count < 1"
31 |       >
32 |         {{ submitting ? 'Borrowing...' : 'Confirm borrow' }}
33 |       </button>
34 |     </div>
35 |   </form>
36 | </template>
37 |
38 | <script setup>
39 | import { ref } from 'vue'
40 | import http from '@/api/http'
41 |
42 | const props = defineProps({
43 |   book: { type: Object, required: true }
44 | })
45 | const emit = defineEmits(['borrowed', 'cancel'])
46 |
47 | const durationDays = ref(14)
48 | const agreed = ref(false)
49 | const submitting = ref(false)
50 | const error = ref('')
51 |
52 | async function submit () {
53 |   error.value = ''
54 |   submitting.value = true
55 |   try {
56 |     const { data } = await http.post('/api/borrow', { book_id: props.book.id })
57 |     emit('borrowed', data)
58 |   } catch (err) {
59 |     error.value = err.response?.data?.message ?? 'Borrow failed. Please try again.'
60 |   } finally {
61 |     submitting.value = false
62 |   }
63 | }
64 | </script>
65 |
66 | <style scoped>
67 | .terms-row {
68 |   display: flex;
69 |   align-items: center;
```

```
70 |     gap: 0.5rem;
71 |     font-size: 0.9rem;
72 | }
73 | .hint { font-size: 0.8rem; margin-top: 0.25rem; }
74 | </style>
```

21. Frontend: Search Bar component

Path: frontend/src/components/SearchBar.vue Lines: 82

```
1 | <template>
2 |   <div class="search-bar">
3 |     <span class="search-icon">■</span>
4 |     <input
5 |       v-model="query"
6 |       type="text"
7 |       class="search-input"
8 |       :placeholder="placeholder"
9 |       @input="emitSearch"
10 |     />
11 |     <button
12 |       v-if="query"
13 |       class="clear-btn"
14 |       type="button"
15 |       aria-label="Clear search"
16 |       @click="clear"
17 |     >x</button>
18 |   </div>
19 | </template>
20 |
21 | <script setup>
22 | import { ref } from 'vue'
23 |
24 | defineProps({
25 |   placeholder: { type: String, default: 'Search books by title, author or category...' }
26 | })
27 |
28 | const emit = defineEmits(['search'])
29 | const query = ref('')
30 |
31 | // Tiny debounce so we don't spam parent on every keystroke
32 | let timer
33 | function emitSearch () {
34 |   clearTimeout(timer)
35 |   timer = setTimeout(() => emit('search', query.value.trim()), 150)
36 | }
37 |
38 | function clear () {
39 |   query.value = ''
40 |   emit('search', '')
41 | }
42 | </script>
43 |
44 | <style scoped>
45 | .search-bar {
46 |   position: relative;
47 |   display: flex;
48 |   align-items: center;
49 |   background: var(--surface);
50 |   border: 1px solid var(--border);
51 |   border-radius: var(--radius);
52 |   padding: 0 0.75rem;
53 |   transition: border-color 0.15s, box-shadow 0.15s;
54 | }
55 | .search-bar:focus-within {
56 |   border-color: var(--primary);
57 |   box-shadow: 0 0 0 3px var(--primary-soft);
58 | }
59 |
60 | .search-icon { font-size: 0.95rem; color: var(--text-muted); }
61 |
62 | .search-input {
63 |   flex: 1;
64 |   border: none;
65 |   outline: none;
66 |   padding: 0.65rem 0.5rem;
67 |   font-size: 0.95rem;
68 |   background: transparent;
69 |   color: var(--text);
```

```
70 | }
71 |
72 | .clear-btn {
73 |   background: none;
74 |   border: none;
75 |   color: var(--text-muted);
76 |   font-size: 1.5rem;
77 |   line-height: 1;
78 |   cursor: pointer;
79 |   padding: 0 0.25rem;
80 | }
81 | .clear-btn:hover { color: var(--text); }
82 | </style>
```

22. Frontend: Loading Spinner component

Path: frontend/src/components/LoadingSpinner.vue Lines: 41

```
1 | <template>
2 |   <div class="spinner-wrap" :class="{ inline }">
3 |     <div class="spinner" :style="{ width: size + 'px', height: size + 'px' }"></div>
4 |     <span v-if="label" class="label">{{ label }}</span>
5 |   </div>
6 | </template>
7 |
8 | <script setup>
9 | defineProps({
10 |   size: { type: Number, default: 24 },
11 |   label: { type: String, default: '' },
12 |   inline: { type: Boolean, default: false }
13 | })
14 | </script>
15 |
16 | <style scoped>
17 | .spinner-wrap {
18 |   display: flex;
19 |   align-items: center;
20 |   justify-content: center;
21 |   gap: 0.5rem;
22 |   padding: 1.5rem 0;
23 | }
24 | .spinner-wrap.inline { padding: 0; }
25 |
26 | .spinner {
27 |   border: 3px solid var(--surface-muted);
28 |   border-top-color: var(--primary);
29 |   border-radius: 50%;
30 |   animation: spin 0.8s linear infinite;
31 | }
32 |
33 | .label {
34 |   font-size: 0.9rem;
35 |   color: var(--text-muted);
36 | }
37 |
38 | @keyframes spin {
39 |   to { transform: rotate(360deg); }
40 | }
41 | </style>
```